

markpublish Benutzerhandbuch

Moderne PDF- und HTML-Dokumentenerstellung aus Markdown

Dieses Handbuch bietet eine praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, Template-Anpassung und Dokumentengenerierung mit markpublish.

Version	1.0.0
Datum	02.09.2026
Autor	Frank Winter
Copyright	© 2026 Frank Winter

Inhaltsverzeichnis

Das Template- und Design-System	4
Theme-basierte Verzeichnisstruktur	4
Die 3-stufige Auflösungs-Hierarchie	4
Statische Texte und Sprache	5
Kopf- und Fußzeilen	9

KAPITEL 5

Das Template- und Design-System

3-stufige Auflösungshierarchie, CSS Paged Media und 2-zeilige Kopf-/Fußzeilen.

Das Template- und Design-System

Das Template-System von `markpublish` ermöglicht vollständige visuelle Anpassbarkeit bei gleichzeitiger Wartbarkeit.

Theme-basierte Verzeichnisstruktur

Templates sind nach Theme gruppiert, darunter liegt das Ausgabeformat. So gehören alle Dateien eines Designs zusammen und ein Theme lässt sich als Ganzes kopieren, weitergeben oder versionieren:

```
templates/
├── default/                                # Theme-Name
│   ├── pdf/
│   │   ├── layout.html                    # HTML-Grundgerüst
│   │   ├── styles.css                    # CSS Paged Media & Kopf-/Fußzeilen
│   │   ├── fonts/                        # Mitgelieferte Schrift (Open Sans, variabel) + OFL.txt
│   │   ├── cover.html                    # Deckblatt-Template
│   │   ├── part_divider.html              # Trennseite für übergeordnete Parts
│   │   ├── chapter_divider.html          # Trennseite für Kapitel mit Mini-TOC
│   │   └── toc.html                       # Globales Inhaltsverzeichnis
│   └── html/
│       ├── layout.html                    # Standalone Web-Layout
│       ├── styles.css                     # Screen/Responsive Stylesheet
│       └── ...
```

Die 3-stufige Auflösungs-Hierarchie

Beim Suchen nach einem Template (z. B. `default/pdf`) gilt folgende strikte Priorität:

1. User-Verzeichnis (`~/markpublish/templates/default/pdf/`)
| (höchste Priorität)
▼
2. Gemeinsamer / Projekt-Ordner (`<templates_dir>/default/pdf/`)
|
▼
3. Paket Built-in (im `markpublish` Python-Paket)

Konfiguration des gemeinsamen Template-Ordners

Sie können den gemeinsamen Template-Pfad auf 4 Wegen festlegen:

1. **CLI-Parameter:** `markpublish build --templates-dir /pfad/zu/templates`
2. In `markpublish.yaml`: `templates_dir: "/pfad/zu/templates"`
3. **Umgebungsvariable:** `MARKPUBLISH_TEMPLATES_DIR=/pfad/zu/templates`

4. **Automatischer Fallback:** `./templates` im Projektordner.

Statische Texte und Sprache

Die festen Beschriftungen — Überschrift des Inhaltsverzeichnisses, Kapitel-Marken, Cover-Labels, Seitenzahl-Fußzeile, Callout-Titel — stehen nicht im Template, sondern in `i18n.yaml`-Dateien. Welche Sprache gilt, bestimmt `document.language`:

```
document:
  title: "User Guide"
  language: "en"      # steuert Beschriftungen, Callout-Titel und Datumsformat
```

Mitgeliefert sind `de` und `en`. Regionale Formen werden zugeordnet (`de-AT` → `de`), eine unbekannte Sprache fällt auf Englisch zurück. Fehlt `language:` ganz, gilt die Systemsprache — `markpublish init` schreibt sie gleich hin, damit ein Dokument auf jedem Rechner gleich gebaut wird.

Hinweis

`i18n` bezeichnet die Quellen — die Dateien und den YAML-Eintrag, jeweils über alle Sprachen. `labels` ist das daraus aufgelöste Ergebnis für ein Dokument in einer Sprache: das, was im Template unter `{{ labels.chapter }}` ankommt.

Die 3-stufige i18n-Kaskade

Alle drei Ebenen sind **identisch aufgebaut**: Sprachcode auf oberster Ebene, darunter die Texte. Jede tiefere Ebene überschreibt die höher liegende — und zwar **nur die Schlüssel, die sie tatsächlich setzt**. Alles andere bleibt, wie es weiter oben steht:

1. Programm	<code>markpublish/i18n.yaml</code> vollständig, <code>de</code> + <code>en</code>
2. Theme	<code><templates>/<theme>/i18n.yaml</code> gilt für alle Zielformate des Themes
3. Zielformat	<code><templates>/<theme>/<target>/i18n.yaml</code> nur für PDF bzw. nur für HTML

Ebene 1 ist die einzige, die vollständig sein muss. Sie legt zusätzlich Englisch unter die Dokumentensprache, damit jeder Programmtext garantiert auflöst. Die Ebenen 2 und 3 sind reine Overrides.

Statische Texte werden **ausschließlich im Theme** definiert. Eine Dokumentenebene gibt es nicht: `document.i18n` in der `markpublish.yaml` wird abgelehnt, mit Hinweis auf die Theme-Datei. Wer die Texte eines Dokuments ändern will, gibt ihm sein eigenes Theme — `markpublish export-template` legt es an.

 Wichtig

Die Ebenen 2 und 3 stammen immer aus **genau einem** Theme: welches gilt, entscheidet vorher die Template-Auflösung (User > Projekt > Paket). Ein Projekt-Theme erbt **nicht** die Texte des gleichnamigen Paket-Themes — gemischt wird nur das eine gefundene Theme mit dem Programmstandard.

 Wichtig

Die Ebenen 2 und 3 greifen **nur für die gewählte Sprache**. Ein Theme, das einen `en: -Block` definiert, verändert eine deutsche Ausgabe nicht — sonst würden englische Theme-Texte in fremdsprachige Dokumente durchschlagen.

Das gemeinsame Format

```
# templates/mytheme/i18n.yaml
de:
  part: "Abschnitt"
  chapter_toc_title: "Auf dieser Seite"
en:
  part: "Section"
  chapter_toc_title: "On this page"
```

Der Sonderschlüssel `"*"` gilt für **jede** Sprache und wird vor dem sprachspezifischen Block angewendet — für Begriffe, die unabhängig von der Dokumentsprache gleich heißen:

```
"*":
  version: "Rev."      # in jeder Sprache "Rev."
de:
  part: "Abschnitt"
```

Wer nur eine Sprache pflegt, darf die Sprachebene auch weglassen; die flache Form ist gleichbedeutend mit `"*"` :

```
part: "Abschnitt"      # entspricht:  "*":\n  part: "Abschnitt"
```

Regionale Blöcke schlagen den Basis-Block: bei `language: "de-AT"` gewinnt `de-at:` über `de:`. Fehlt eine `i18n.yaml`, ist das kein Fehler — ein Theme ohne eigene Texte ist der Normalfall. Ist sie vorhanden, aber fehlerhaft, bricht der Build mit Angabe der Datei ab, statt still die Standardtexte zu verwenden.

Beispiel: PDF und HTML unterschiedlich beschriften

```

templates/mytheme/
├── i18n.yaml          de: chapter: "Kapitel"
├── pdf/
│   └── i18n.yaml      de: chapter: "Kap."    ← nur im PDF
└── html/
    └── i18n.yaml      (leer → erbt "Kapitel")

```

Das mitgelieferte Theme `default` bringt alle drei Dateien als **auskommentiertes Muster** mit: `templates/default/i18n.yaml`, `default/pdf/i18n.yaml` und `default/html/i18n.yaml`. Sie sind bewusst wirkungslos — das Standard-Theme soll exakt wie der Programmstandard aussprechen. Kommentieren Sie aus, was Sie ändern möchten. `markpublish export-template` kopiert diese Dateien mit — auch die `i18n.yaml` der Theme-Ebene, die neben den Zielformat-Ordnern liegt.

Freie Labels: eigene Texte des Templates

Ein Theme darf **eigene Schlüssel** definieren, die das Programm nicht kennt — für die statischen Texte des Templates selbst. Der Aufbau ist derselbe, der Zugriff ebenso:

```

# templates/mytheme/i18n.yaml
de:
  imprint_title: "Impressum"
  disclaimer: "Alle Angaben ohne Gewähr."
en:
  imprint_title: "Imprint"
  disclaimer: "All information without guarantee."

```

```

<!-- templates/mytheme/html/layout.html -->
<footer>{{ labels.imprint_title }}</footer>

```

Warnung

Für freie Labels gibt es **keinen Programmstandard**, der einspringen könnte. Deshalb gilt hier eine strikte Regel: Ein Label, das ein Template notiert, **muss** in der Kaskade auflösen. Tut es das nicht, bricht der Build ab und nennt Schlüssel, Fundstelle mit Zeilennummer, Dokumentsprache und die durchsuchten Dateien:

```

Label 'imprint_title' ist im Template notiert, aber in keiner i18n-Ebene definiert.
Dokumentsprache: de
Fundstelle:
templates/mytheme/html/layout.html:108
Gesucht in:
templates/mytheme/html/i18n.yaml (vorhanden)
templates/mytheme/i18n.yaml      (vorhanden)
markpublish/i18n.yaml            (vorhanden)

```

Pflegen Sie also jede Sprache, die Sie ausliefern, oder legen Sie den Schlüssel unter `"*"` ab. Ein leerer Text im fertigen PDF fällt niemandem auf — ein Abbruch schon.

Geprüft werden die Template-Quellen, nicht der Renderlauf: auch ein Label in einem Zweig, den genau dieses Dokument nicht durchläuft, wird gemeldet. `styles.css` zählt mit, da es durch dieselbe Jinja-Umgebung läuft.

Die aufgelöste Tabelle ansehen

Bei drei Ebenen ist nicht immer offensichtlich, woher ein Text kommt. `markpublish labels` zeigt das Ergebnis samt Herkunft:

```
markpublish labels          # alle Schlüssel, Ziel PDF
markpublish labels --target html # Kaskade für die HTML-Ausgabe
markpublish labels --overridden # nur das, was vom Theme kommt
```

Verfügbare Schlüssel

Schlüssel	de	en
<code>toc_title</code>	Inhaltsverzeichnis	Table of Contents
<code>toc_sidebar</code>	Inhalt	Contents
<code>chapter_toc_title</code>	Inhalt dieses Kapitels	In this chapter
<code>chapter</code>	Kapitel	Chapter
<code>part</code>	Teil	Part
<code>author</code>	Autor	Author
<code>status</code>	Status	Status
<code>version</code>	Version	Version
<code>date</code>	Datum	Date
<code>copyright</code>	Copyright	Copyright
<code>page</code>	Seite	Page
<code>page_of</code>	von	of
<code>alert_note</code>	Hinweis	Note
<code>alert_tip</code>	Tipp	Tip
<code>alert_important</code>	Wichtig	Important
<code>alert_warning</code>	Warnung	Warning
<code>alert_caution</code>	Achtung	Caution

Die `alert_*`-Titel entstehen bereits beim Markdown-Parsen, nicht erst im Template — die Kaskade wird dorthin durchgereicht, ein `alert_note` im Theme wirkt also auch im Callout.

Eine neue Sprache legen Sie an, indem Sie in `markpublish/i18n.yaml` einen vollständigen Block ergänzen — oder, ohne das Paket anzufassen, auf Ebene 2 oder 3 alle Schlüssel unter dem gewünschten Sprachcode setzen.

Tipp

In eigenen Templates greifen Sie mit `{{ labels.chapter }}` auf das Ergebnis zu — auch in `styles.css`, das durch dieselbe Jinja-Umgebung läuft. So ist die Fußzeile `"{{ labels.page }}" counter(page) " {{ labels.page_of }}" counter(pages)` gebaut.

Kopf- und Fußzeilen

Kopf- und Fußzeile sind **Running Elements**: ein Block im `layout.html` bekommt `position: running(name)` und wird damit aus dem Textfluss genommen; die `@page`-Regel setzt ihn über `content: element(name)` in die Margin-Box ein.

Der Unterschied zu einer `content:`-Zeichenkette ist wesentlich: In der Margin-Box steht dadurch **echtes Markup**. Damit sind beliebig viele Zeilen möglich, jede mit eigener Auszeichnung — eine Zeichenkette kennt nur eine Formatierung für alles.

```
<!-- layout.html -->
<div class="page-header-runner">
  <div class="hf-left">
    <div class="hf-doc-title">{{ document.title }}</div>
    <div class="hf-doc-subtitle">{{ document.subtitle }}</div>
  </div>
  <div class="hf-right"><span class="hf-section"></span></div>
</div>
```

```
/* styles.css */
.page-header-runner {
  position: running(pageheader);
  display: flex;
  justify-content: space-between;
  align-items: flex-start; /* rechte Spalte bleibt oben */
}
.hf-doc-title { font-weight: 700; }
.hf-section::before { content: string(current-section); }

@page {
  @top-left {
    content: element(pageheader);
    width: 100%;
    vertical-align: top;
  }
}
```

```

margin-top: 12mm;           /* Abstand zur Papierkante */
padding-bottom: 0;
border-bottom: 0.5pt solid #cbd5e1;
margin-bottom: 12mm;       /* Abstand zum Inhalt */
}
}

```

Geometrie

Von der Papierkante nach innen:

```

Kante — margin — Zeilen (top-aligned) — Linie — margin — Inhalt
      12 mm                               12 mm

```

Beide Abstände sind bewusst gleich groß: eine Kopfzeile, die dicht über dem Text sitzt, liest sich als Teil des Satzspiegels statt als Seitenfurnitur.

Beide Spalten sitzen in einem Flex-Container mit `align-items: flex-start`. Die rechte Spalte beginnt deshalb auf der Höhe der **ersten** linken Zeile, auch wenn links drei Zeilen stehen und rechts nur eine.

Wichtig

Der Abstand zum Inhalt kommt aus `margin-bottom`, nicht aus `padding-bottom`. In CSS liegt der Rahmen **außerhalb** des Paddings — ein `padding-bottom` würde die Trennlinie vom Text wegschieben und an den Inhalt drücken. Gewollt ist das Gegenteil: Linie direkt am Text, Abstand danach.

Warnung

Eine Margin-Box **schiebt den Inhalt nicht**. Ihre Höhe ist durch den Seitenrand gedeckelt; zusätzliche Zeilen laufen aus der Seite heraus, statt den Satzspiegel zu verkleinern. Der Seitenrand wird deshalb in `styles.css` aus der Zeilenzahl gerechnet:

```
margin-top = Rand zur Kante + Zeilen × Zeilenhöhe + Linienstärke + Abstand zum Inhalt
```

Wer in `layout.html` eine Zeile ergänzt, zieht `hf_header_lines` bzw. `hf_footer_lines` in `styles.css` mit.

Standardbelegung des Themes

Kopfzeile	Dokumenttitel (fett) Untertitel	Kapiteltitel
Fußzeile	Copyright	Version 1.0.0 01.09.2026 Seite X von Y

Untertitel und Version sind optional. Fehlt der Untertitel, hat die Kopfzeile nur eine Zeile und der Satzspiegel rückt entsprechend nach oben. Fehlt die Version, entfällt sie samt Trenner — in der Fußzeile steht dann nur das Datum, und auf dem Deckblatt fehlt das Feld ganz.

Der Kapiteltitel kommt aus `string(current-section)`, das die Kapitel- `<article>` per `string-set` setzen; `counter(page)` funktioniert innerhalb des Running Elements ebenso wie in einer Margin-Box. Die Schalter `document.header` und `document.footer` blenden den jeweiligen Block ab; auf Deck- und Trennseiten ist er ohnehin abgeschaltet.